

coordinate_conversion.py

Code Flow & Architecture Documentation

How a LINE from (0,0) to (5,0) gets converted from UTM metres to WGS84 degrees

Source CRS	EPSG:32642 — UTM Zone 42N (metres)
Target CRS	EPSG:4326 — WGS84 lat/lon (degrees)
Key libraries	pyproj (coordinate math) ezdxf (DXF file I/O) os (file paths)

1. Overview

This document traces the exact code path that coordinate_conversion.py follows when converting a DXF entity from UTM Zone 42N (metres) to WGS84 lat/lon (degrees). Every function call, every variable, and every library involved is documented in order.

The file contains four layers of code, each with one clear job:

- convert_dxf_file() — the manager. Reads the file, loops over shapes, saves output.
- transform_xy() — the router. Picks forward (UTM → latlon) or reverse direction.
- utm_to_latlon() / latlon_to_utm() — the wrappers. Call the Transformer and round the result.
- Transformer.transform() — the engine. All projection math, handled by pyproj / PROJ C library.

No layer talks to a layer more than one level below it. convert_dxf_file never directly calls forward_transformer — it always goes through transform_xy first. This is called separation of concerns.

2. Libraries involved

Library	Install needed?	Job in this file	Does math?
pyproj	Yes — pip install pyproj	Converts coordinate numbers between CRS systems	YES — all of it
ezdxf	Yes — pip	Reads and writes DXF files, exposes	No

	install ezdxf	entities as Python objects	
os	No — built into Python	File path utilities — check existence, split extension	No

3. Step-by-step code flow

The example input is a single LINE entity: start (0, 0, 0) and end (5, 0, 0) in UTM Zone 42N metres. The following steps trace exactly what happens in the code.

Step 1	Module level — runs once when file loads Before any function is called, Python executes the top-level code. Two Transformer objects are created and stored as module-level variables. These are built once and reused for every entity. <pre>forward_transformer = Transformer.from_crs("EPSG:32642", "EPSG:4326", always_xy=True) reverse_transformer = Transformer.from_crs("EPSG:4326", "EPSG:32642", always_xy=True) # Result: two configured objects in memory, ready to use</pre>
Step 2	convert_dxf_file() is called This is the main function. It reads the DXF file using ezdxf, creates an empty output document, copies layers and linetypes, then loops over every entity in the modelspace. <pre>doc = ezdxf.readfile(input_file) msp = doc.modelspace() output_doc = ezdxf.new(dxfversion=doc.dxfversion) output_msp = output_doc.modelspace() for entity in msp: # ← loop enters here</pre>
Step 3	Loop hits the LINE entity entity.dxf.type() returns "LINE" so the matching elif block runs. The start and end coordinates are unpacked into variables. <pre>elif entity.dxf.type() == 'LINE': x1, y1, z1 = entity.dxf.start # → 0, 0, 0 x2, y2, z2 = entity.dxf.end # → 5, 0, 0 new_x1, new_y1 = transform_xy(x1, y1, reverse=False) new_x2, new_y2 = transform_xy(x2, y2, reverse=False)</pre>
Step 4	transform_xy() — the router This tiny function has one job: decide which direction to convert. Since reverse=False, it calls utm_to_latlon(). It does no math itself. <pre>def transform_xy(x, y, reverse=False): if reverse: return latlon_to_utm(x, y) # skipped — reverse is False return utm_to_latlon(x, y) # ← this runs</pre>
Step 5	utm_to_latlon() — calls the Transformer

	This function calls <code>forward_transformer.transform()</code> — the object created in Step 1. <code>pyproj</code> runs the full transverse Mercator inverse projection internally and returns (longitude, latitude). <pre>def utm_to_latlon(x, y): lon, lat = forward_transformer.transform(x, y) # For (0, 0): returns lon=64.50660, lat=0.0 # For (5, 0): returns lon=64.50711, lat=0.0 return round(lon, 8), round(lat, 8)</pre>
Step 6	Both points returned — new LINE written Back in <code>convert_dxf_file()</code>, both converted points are used to create a new LINE entity in the output modelspace. Z is copied unchanged. The layer name is preserved. <pre>new_entity = output_msp.add_line((64.50660, 0.0, 0), # start - lon, lat, z (64.50711, 0.0, 0) # end - lon, lat, z) new_entity.dxf.layer = entity.dxf.layer created_count += 1</pre>
Step 7	File saved to disk After all entities are processed, <code>ezdxf</code> writes the output document to disk. The message reports how many entities were found vs converted. <pre>output_doc.saveas(output_file) # Returns: (True, "Conversion complete! Found: 1 Converted: 1", output_file)</pre>

4. Complete call stack

Reading top to bottom — who called who, for a single point (0, 0):

```
convert_dxf_file()           # [1] main function
└─ transform_xy(0, 0)        # [2] router – picks direction
    └─ utm_to_latlon(0, 0)    # [3] conversion wrapper
        └─ forward_transformer # [4] Transformer object (pyproj)
            .transform(0, 0)   # calls PROJ C library internally
                                # returns (64.50660, 0.0)
                                # (64.50660, 0.0)
        ← round() applied      # (64.50660, 0.0)
    ← returned to transform_xy # (64.50660, 0.0)
← returned to convert_dxf_file # new_x1=64.50660, new_y1=0.0
```

The same path is followed for the second point (5, 0), returning (64.50711, 0.0). Both results are then written as a new LINE entity.

5. Function and class summary

Function / Object	Type	Job	Does Math?
convert_dxf_file()	Function	Manager — reads file, loops entities, saves output	No
transform_xy()	Function	Router — picks forward or reverse direction	No
utm_to_latlon()	Function	Calls forward_transformer, rounds result	No
latlon_to_utm()	Function	Calls reverse_transformer, rounds result	No
Transformer (pyproj)	Class	Configured math engine — wraps PROJ C library	YES — all of it
.from_crs()	Class method	Builds the transformer from two EPSG codes	Setup only
.transform()	Instance method	Runs the actual coordinate conversion	YES

6. Shape conversion fidelity

Not all DXF entity types are converted with the same accuracy. The table below shows what happens to each supported shape type.

Shape	What is transformed	Status	Known limitation
POINT	X, Y → converted. Z copied.	Fully accurate	None
LINE	Both endpoints converted. Z copied.	Fully accurate	None
TEXT	Insertion point converted. Content/height copied.	Fully accurate	None
LWPOLYLINE	Every vertex converted.	Partial	Bulge (arc) values dropped — curves become straight lines
POLYLINE	Vertices converted. Written as LWPOLYLINE.	Partial	3D flattened to 2D. Arcs lost.
CIRCLE	Centre converted. Radius copied unchanged.	Partial	Radius stays in metres — wrong unit in degree space
ARC	Centre converted. Radius + angles copied.	Partial	Radius wrong units. Shape distorted.
SPLINE / HATCH / INSERT / others	No handler written.	Dropped	Silently skipped — missing from output entirely

Fully accurate — position perfectly converted. **Partial** — position correct, shape/size has known error. **Dropped** — entity missing from output file entirely.

7. Key concepts for beginners

What is a class?

A class is a blueprint that bundles data and functions together. Transformer is a class from pyproj. You build it once with `Transformer.from_crs()` — which looks up both CRS definitions, finds the best mathematical path between them, and compiles it. After that, `.transform()` is cheap to call millions of times.

What is always_xy=True?

EPSG:4326 (WGS84) officially defines its axes as (latitude, longitude) — i.e. (Y, X). But engineers always think (X, Y) = (longitude, latitude). The `always_xy=True` flag forces every CRS to use (X, Y) order. Without it, passing (lon, lat) gets silently interpreted as (lat, lon), producing a point thousands of kilometres away with no error.

Why are transformers created at module level?

Building a Transformer involves a database lookup and pipeline compilation — it takes a few milliseconds. Calling `.transform()` is nearly instant. By creating both transformers once at the top of the file (outside any function), they are built once and reused for every entity in the loop, instead of being rebuilt thousands of times.

Why does Z stay in metres?

Z (elevation) is not part of the horizontal CRS transformation. EPSG:32642 and EPSG:4326 both define only the horizontal plane. Elevation is handled separately by a vertical CRS. The code copies Z unchanged — which is the standard accepted trade-off for DXF coordinate conversion.

8. Numeric result for the example

Point	Input (UTM metres)	Output (WGS84 degrees)	Note
Start	(0, 0, 0)	(64.50660, 0.0, 0)	64.5°E, equator
End	(5, 0, 0)	(64.50711, 0.0, 0)	5 m east → +0.00051°

The 5-metre horizontal distance in UTM becomes a 0.00051° difference in longitude. At the equator, $1^\circ \approx 111,320$ m, so $0.00051^\circ \approx 56.8$ m — larger than 5 m because the example point is 500 km from the zone centre where UTM distortion is significant. In a real survey file near Easting 500,000, the result would be the expected 0.0000449°.

